Applied Soft Computing Journal 186 (2026) 114100

Contents lists available at ScienceDirect

Applied Soft Computing

journal homepage: www.elsevier.com/locate/asoc

Complex network based machine learning method for predicting circuit timing

Tingyuan Nie*, Mingzhi Zhao, Pengfei Liu, Zhenhao Wang

Qingdao University of Technology, Qingdao, Shandong, China

HIGHLIGHTS

- Propose a complex network-based machine learning method to predict circuit timing.
- Basic and augmented models are constructed based on circuit and complex network features.
- Augmented model outperforms basic model in timing prediction accuracy.
- It demonstrates the efficiency of utilizing complex networks for predicting circuit timing.

ARTICLE INFO

Keywords:
Static timing analysis
Complex network
Machine learning
Layout
Routing

ABSTRACT

Due to the high cost of timing signoff in the routing stage of VLSI (very large-scale integration), timing analysis in the placement stage is usually used to assist the implementation. Still, uncertainty exists due to the lack of detailed wiring information. Considering the close relationship between signal propagation and circuit topology, we propose a machine-learning method to predict the circuit timing using complex network features and construct basic and augmented models for comparison. Firstly, in the placement stage, the fundamental circuit features related to timing optimization are extracted, and the complex network features on the timing path are computed by complex network modeling, based on which the dataset for machine learning is generated. Secondly, we use machine learning algorithms to compare the performance of the two models in predicting circuit timing. Experimental results show that the augmented model combining complex network features outperforms the basic model in timing prediction accuracy. The augmented model achieves a 12.5 % reduction in MSE (mean squared error) for predicting slack, reaching 0.004515. It also shows a 1.06 % improvement in R^2, reaching 0.9314, and a 0.54 % improvement in r, reaching 0.9652. For predicting delay, the MSE is reduced by 11.69 %, reaching 0.000302, while R^2 improves by 0.03 % to 0.9973, and r improves by 0.02 % to 0.9987. The deviation in predicting WNS (worst negative slack) is reduced by 6.53 %, reaching 5.30 %, and for TNS (total negative slack), it is reduced by 1.59 %, reaching 12.38 %. The AUC (area under the ROC (receiver operating characteristic) curve) value for identifying critical paths increases by 0.26 %, reaching 0.9873. These results demonstrate the efficiency of the proposed method in utilizing complex network features for predicting circuit timing.

1. Introduction

Timing convergence in very large-scale integration (VLSI) design is one of the key challenges in achieving the expected operating frequencies and delay constraints in circuit requirements [1,2]. Static timing analysis (STA) is one of the most common techniques used in digital design verification to resolve various timing issues and accelerate timing convergence in chip design tasks [3].

Designers employ timing analysis to evaluate circuit timing performance, ensuring that the circuit's timing behavior meets design requirements. The accuracy and efficiency of this process significantly impact chip quality and cost. If post-routing timing analysis reveals violations, designers must modify and optimize circuits until timing convergence is achieved. To conserve substantial design resources during the design process, timing analysis in the layout phase becomes critical.

* Corresponding author.
Email addresses: tynie@qut.edu.cn (T. Nie), zhao17662839186@163.com (M. Zhao), 17852849913@163.com (P. Liu), wangzhenhao@qut.edu.cn (Z. Wang).

https://doi.org/10.1016/j.asoc.2025.114100
Received 24 October 2023; Received in revised form 3 May 2025; Accepted 10 October 2025
Available online 31 October 2025
1568-4946/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

However, there is still considerable uncertainty in the timing prediction due to the lack of routing information at the placement stage [4].

Machine learning has made notable advancements in circuit timing analysis in recent years. Barboza et al. [5] proposed a method to predict signal wire delays and transition rates in the routing phase by training a wire delay model and obtaining overall circuit timing through PERT traversal [6]. As it underestimated the wire capacitance in post-routing circuits, this method yielded favorable estimates only for specific test designs. Xie et al. [7] introduced a technique using customizable graph attention networks to predict wire lengths and timing during the circuit layout phase. Guo et al. [8] presented a heuristic-driven temporal engine graph neural network approach for predicting arrival times and Slack of post-routing timing paths.

Cheng et al. [9] suggested a timing prediction method based on RC network modeling, focusing on logic gate inputs and outputs on signal lines in cases of short paths, few nodes, and simple structures. Chen et al. [10] proposed the first post-routing timing prediction model using path-based analysis (PBA), treating timing paths within circuits as the analytical subject and comprehensively considering timing characteristics. However, the approach did not account for circuit topology, which reduced its predictive accuracy.

Complex networks, serving as mathematical tools to effectively describe and study complex systems, can portray relationships and structural features among elements within complex systems, and they find widespread application in complex system studies. Pei et al. [11] proposed a complexity theory-based non-uniform hydrological sequence mutation diagnostic method to forecast hydrological information. Cui et al. [12] addressed interaction behaviors generated during software execution in complex networks and proposed a method to detect abnormal software interaction behaviors in directed complex networks. Rezaei et al. proposed a machine learning-based approach for vital node identification in complex networks and demonstrated its efficiency [13].

Integrated circuits consist of numerous electronic components and intricate interconnect structures, which interact to produce complex functions and behaviors and are regarded as complex systems. Currently, the application of complex network theory to integrated circuits is increasing. Cancho et al. [14] modeled circuits using complex networks to analyze average shortest paths and clustering coefficients and identified small-world properties in analog circuits. Nie et al. [15] introduced a routing congestion prediction method based on complex network knowledge, and a patched EDM (elucidating the design space of a diffusion-based generative model) with a patch transformation was introduced to focus on the most significant features. Subsequently, they predicted the routability of FPGA designs by learning complex network images. The evaluation highlights the classification efficiency of the proposal [16].

Therefore, this paper incorporates the concept of complex networks into the predictive circuit timing model. It extracts complex network feature parameters and post-layout timing features during the circuit layout phase, obtains post-routing timing features during the routing phase, aggregates them into a dataset based on timing paths, and presents a machine learning approach that utilizes complex network features to predict post-routing circuit timing.

The organization of this paper is as follows: Section 2 introduces complex network features and circuit timing features, Section 3 explains the experimental methodology, and Section 4 analyzes and interprets the experimental results. Lastly, Section 5 summarizes the work presented in this paper and discusses future endeavors.

2. Feature introduction and basic concepts

This paper uses circuit timing and complex network features for placement to predict post-routing circuit timing. The aim is to capture complex timing behavior within circuits more accurately. This chapter introduces the fundamental principles of circuit timing and complex network features.

2.1. Complex network features

Complex network features play a significant role in the machine learning-based circuit timing prediction method. This section introduces the main complex network concepts used in machine learning.

2.1.1. Degree strength

Degree strength is a metric used to measure the strength of connections of a node in a network. It is defined as the weighted sum of edges between a node and other nodes. The calculation formula is as follows.

$$D_S(i) = \sum_{j \neq i} W(ij), \tag{1}$$

where i represents a specific node, j represents all neighboring nodes connected to node i, and $W(ij)$ represents the weight of the connection between node i and node j.

2.1.2. Eigenvector centrality

Eigenvector centrality measures the importance of a node in a network, considering the importance of nodes connected to it. It can be calculated using the adjacency matrix and eigenvector solving.

$$A\mathbf{x} = \lambda \mathbf{x}, \tag{2}$$

where A is the adjacency matrix of the network, A_{ij} represents the connection between node i and node j, λ is the corresponding eigenvalue. and x is the eigenvector of the node, a nonzero vector of size n, where each element represents the eigenvector centrality of the corresponding node.

2.1.3. Closeness centrality

Closeness centrality calculates the centrality of a node by summing the reciprocals of its shortest paths to all other nodes in the graph.

$$C_C(v) = \frac{1}{\sum_{v \in V} d(u, v)}, \tag{3}$$

where V represents the set of nodes in the graph, and $d(u, v)$ represents the shortest path length from node u to node v. Closeness centrality $C_C(v)$ of node v indicates how close it is to other nodes; a higher v value signifies closer proximity.

2.1.4. Betweenness centrality

Betweenness centrality measures a node's importance in terms of the number of shortest paths that pass through it.

$$C_B(i) = \sum_{s \neq i \neq t \in V} \frac{\sigma_{st}(i)}{\sigma_{st}}, \tag{4}$$

where σ_{st} represents the number of shortest paths from node s to node t, $\sigma_{st}(i)$ represents the number of shortest paths from node s to node t that pass through node i, and V represents the set of all nodes in the network.

2.1.5. Edge betweenness centrality

Edge betweenness centrality extends betweenness centrality to edges in the network, measuring the importance of edges. It is defined as the sum of the frequencies with which an edge appears in the shortest paths of the network.

$$C_B(e) = \sum_{s \neq t \in V} \frac{\sigma_{st}(e)}{\sigma_{st}}, \tag{5}$$

where $\sigma_{st}(e)$ represents the number of shortest paths from node s to node t that pass through edge e.

Table 1
Circuit features in the layout.

Feature name	Quantity	Description
HPWL	1	"Half perimeter wire length (HPWL)" is used to estimate the timing path length, calculated by the half perimeter wire length between all connected nodes on the timing path and then summed up.
Fanout number	3	The fanout count refers to the number of loads driven by the output signal in a logic gate. This study calculates the maximum, minimum, and average fanout counts of cells in each timing path.
Slack	1	The margin value on the timing path indicates a negative value, indicating the presence of a timing violation, typically underestimated during the layout phase.
Delay	1	The delay value on the timing path is the time required for signals to propagate along the circuit timing path during the layout phase.
Required time and Arrival time	2	Required time refers to the minimum time required for a signal to travel from input to output. Arrival time refers to the time when a signal reaches the output port. In circuit design, it is essential to ensure that the arrival time of each signal is less than or equal to its corresponding required time. Otherwise, it can lead to circuit instability and even failure.
Logic level	1	Logic level indicates the number of logic devices in a timing path.
LUT	6	A look-up table (LUT) is a general digital logic circuit component for computing and storing logic functions. An FPGA (field programmable gate array) design has six different types of LUTs, referred to as LUT1 to LUT6, and the number of each LUT along the timing path.

2.2. Circuit timing features

The basic circuit timing features used in this paper include the results of static timing analysis during the placement phase and detailed information about individual units. Table 1 displays the names, descriptions, and quantities of circuit timing features used in the experiments.

In this work, we do not use cell delay and transition time correlated with transition rate as the features for the prediction. There are two main styles of timing analysis, i.e., graph-based timing analysis (GBA) mode and path-based timing analysis (PBA) mode. GBA mode can quickly obtain timing analysis results. However, the results are often pessimistic because it uses the worst signal transition time across all input pins of the cell to calculate cell delay. PBA uses the specific transition time of each pin to compute the cell delay and propagates the transition time along the path to obtain a relatively accurate delay for each multi-input stage. However, numerous data paths and complex calculations lead to a massive time overhead [17].

A lack of information on cell delay and transition rates could affect the proposal's performance. Instead, we use the calculated maximum, minimum, and average values of the features of the complex network corresponding to the cells along the data path. We expect these parameters to provide comprehensive information and insights for timing prediction as an alternative.

3. Using complex network features for timing prediction

This study uses the circuits from the ISPD 2016 benchmark [18] for the experiment. Combining integrated circuit design, complex network theory, and machine learning techniques through physical design, this study explores timing analysis and the correlation between complex network characteristics and circuit timing. This chapter provides a detailed introduction to the timing prediction method framework, complex network modeling, and the machine learning algorithms and evaluation metrics employed.

3.1. Framework of the timing prediction method

Fig. 1 shows the framework of the proposed method for predicting circuit timing using complex network features. It consists of two main stages: dataset construction at the top and model construction at the bottom of the figure.

The first stage is dataset construction, where the timing features and timing labels are extracted during the placement and routing phases, respectively. The circuits in the benchmark are placed to obtain the placement, which is transformed into a complex network representation through complex network modeling, and complex network features are computed. At the same time, the Vivado tool retrieves the circuit features related to timing analysis. After routing, timing analysis tools extract the

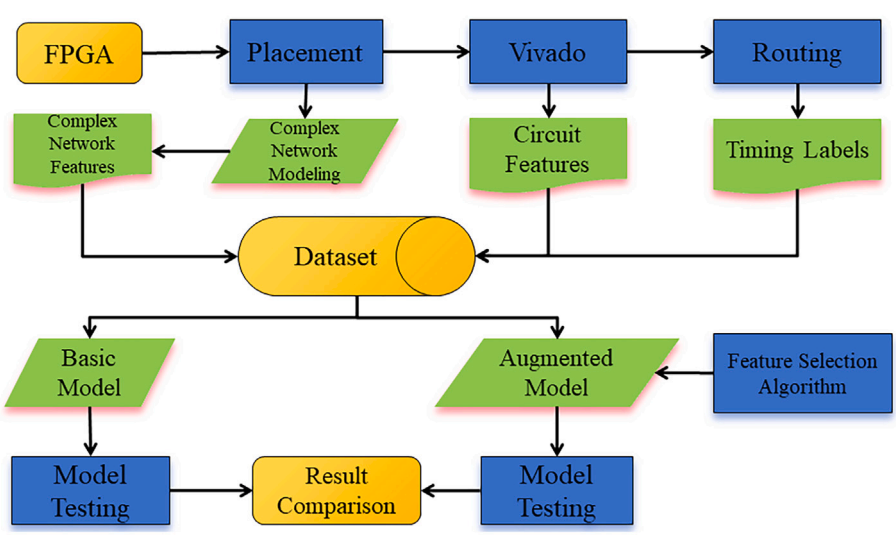

Fig. 1. Framework of the proposed timing prediction method.

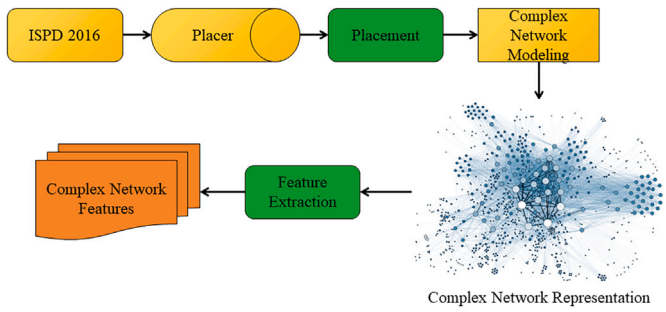

Fig. 2. Complex network modeling.

associated timing metrics that serve as labels for machine learning. The resulting features and labels are aligned based on the timing paths to generate the machine learning dataset.

The second stage is model construction. In this paper, two models are proposed: the basic model and the augmented model. The dataset used to train the basic model includes only fundamental circuit features related to timing optimization. While these features play a role in predicting circuit timing, they lack a comprehensive consideration of the circuit topology. Subsequently, the augmented model is constructed. In addition to the basic circuit features, this model's dataset incorporates complex network features. A feature selection algorithm is used to select highly relevant combinations of features to improve the predictive performance.

Finally, various machine-learning methods are applied to the datasets. The trained models are compared and analyzed using evaluation metrics on the prediction results. This comparison validates the efficiency of the proposed method for predicting circuit timing using complex network features.

3.2. Complex network modeling

At the placement stage, we use complex network modeling to extract complex network features to construct the dataset. We extract module locations and module-to-module connection information from the placement file. This data forms the basis for creating a two-dimensional undirected weighted complex network. In the network representation, the circuit modules are represented as nodes, the connections are represented as edges, and the half-perimeter distance between connected modules is the weight assigned to the edge.

Fig. 2 illustrates the process of the complex network modeling. The circuit is first placed by the placer to retrieve the placement. By applying complex network modeling, the placement is converted into a complex network representation, and complex network feature parameters such as degree strength, eigenvector centrality, betweenness centrality, closeness centrality, and edge betweenness centrality are calculated using Matlab and R software tools. The maximum, minimum, and average values of complex network features corresponding to cells along each timing path are calculated, thus providing more comprehensive information and clues for timing prediction.

3.3. Dataset construction

Fig. 3 illustrates the construction of a dataset for the timing prediction model. This process includes acquisition of the circuit and complex network features during the placement phase and timing labels during post-routing.

The circuit placement in the ISPD 2016 Benchmark is obtained using the placer UTPlaceF [19], and basic circuit features for the placement are extracted. Using the igraph package in R software, complex network feature parameters such as degree strength (DS), eigenvector centrality (EC), betweenness centrality (BC), closeness centrality (CC), and edge betweenness centrality (EBC) are computed from the complex network model generated.

Firstly, the circuit placement in the ISPD 2016 benchmark is retrieved using the placer UTPlaceF [19], and circuit features for the placement are extracted. Complex network modeling generates the complex network for the placement, from which complex network feature parameters such as degree DS, EC, BC, CC, and EBC are computed using the igraph package of R software.

Subsequently, the circuit placement is routed using the Vivado [20] routing tool. Static timing analysis is performed on the circuit to generate path-based timing reports. A timing path refers to the propagation path of a signal from a clock-triggered flip-flop to a target register in digital circuits consisting of a series of logic gates and wires. The work extracts timing path slacks and delays from post-routing timing reports, which serve as labels for the dataset. In addition, labels indicating critical paths are generated based on positive and negative slacks. A path with negative slack is considered a critical path.

Finally, the obtained features and labels are matched according to each timing path, generating a data record for each path. The manipulation is repeated for all the circuits to generate the final dataset.

3.4. Machine learning methods

For model selection, we considered XGBoost and random forest (RF) algorithms, both of which provide feature selection capabilities. XGBoost, an efficient gradient-boosting tree algorithm, is commonly

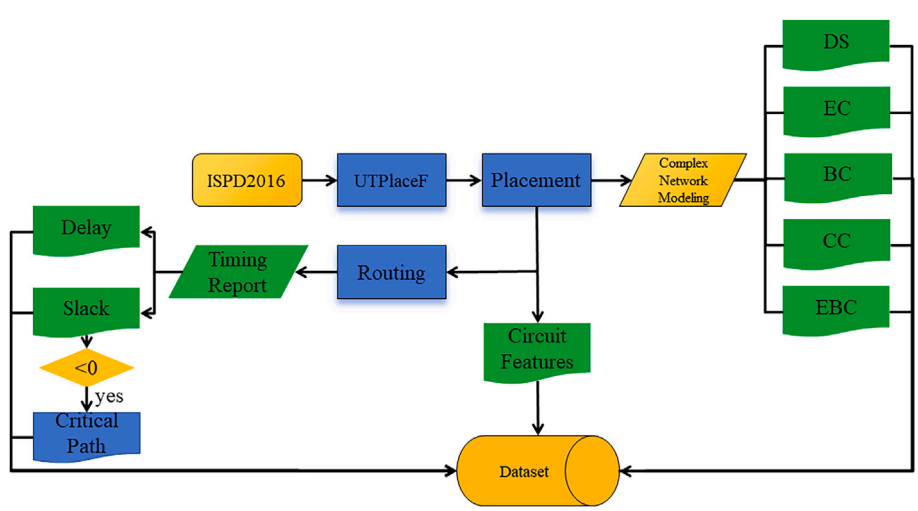

Fig. 3. Dataset construction.

used for regression and classification tasks [21]. It ranks feature importance using built-in functions and performs feature selection based on the calculated feature importance scores.

The RF algorithm uses recursive feature elimination (RFE), which iteratively trains models and removes features with smaller contributions to the model. In this experiment, the RF algorithm showed better overall prediction results. As an ensemble learning algorithm, an RF consists of multiple decision trees. By combining several weak models into one strong model, Random Forest effectively improves the predictive ability [22].

One of the main drawbacks of neural network models is their lack of interpretability. The experimental results in [5] demonstrated the inferior performance of the neural network model in terms of delay and slack prediction. However, the RF algorithm demonstrated greater interpretability during the evaluation process. Therefore, we select the RF model alongside the XGBoost model to evaluate the efficiency of complex network-based circuit timing prediction. We demonstrate the efficiency of the selected models in the experiment.

3.5. Evaluation metrics

The evaluation metrics used in this paper are as follows.

(1) MSE (Mean Squared Error)

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{6}$$

(2) R^2 (The R-squared metric)

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2} \tag{7}$$

(3) r (Correlation coefficient)

$$r = \frac{n\sum_{i=1}^{n} y_i\hat{y}_i - \sum_{i=1}^{n} y_i \sum_{i=1}^{n}\hat{y}_i}{\sqrt{n\sum_{i=1}^{n} y_i{}^2 - \left(\sum_{i=1}^{n} y_i\right)^2}\sqrt{n\sum_{i=1}^{n}\hat{y}_i^2 - \left(\sum_{i=1}^{n}\hat{y}_i\right)^2}} \tag{8}$$

where n is the size of the test set, y_i is the true values and $\hat{y}_i$ is the predicted value of the i-th data point, and $\bar{y}$ is the mean of the true values. MSE reflects the differences between predicted and true values. R-squared (R^2) measures how well the model replicates the observed results, thereby determining the model's accuracy. The correlation coefficient (r) measures the strength and direction of a linear relationship between the predicted and actual values. Both metrics range from -1 to 1, with R^2 being the square of the correlation coefficient and representing the proportion of variance in a dependent variable explained by an independent variable. A biased estimator consistently produces predictions higher or lower than the ground-truth labels. It may achieve high r but much lower R^2 if it correlates with the label well [23].

(4) AUC (Area under the ROC curve)

AUC is a performance metric for evaluating binary classification models. It represents the area under the receiver operating characteristic (ROC) curve. The ROC curve is constructed by varying the classification threshold of a binary classification model, with the true positive rate (TPR) on the y-axis and the false positive rate (FPR) on the x-axis. AUC values range from 0 to 1; values closer to 1 indicate better classification performance.

4. Experimental setup and analysis

In this section, we present the experimental setup and then demonstrate the efficiency of the work through data analysis.

Table 2
Circuit information.

Circuit name	Path number	Nets (K)	Cells (K)	Constraint	LUTs (K)	FF (K)
D1	2664	112	112	13	49	55
D2	2522	456	447	31	245	170
D3	2656	451	442	37	245	172
D4	2130	436	444	44	246	174
D5	2570	885	878	46	467	363
D6	1440	186	184	15	98	74
D7	1057	765	757	40	485	216
D8	1176	454	445	49	246	174

4.1. Experimental setup

A Lenovo workstation with a 2.20 GHz CPU and 16.00 GB of memory running Linux Ubuntu 18.04 was chosen as the platform for the experiment. UTplaceF was used for the placement, while Vivado was used for routing and timing analysis. The machine learning models were implemented using the scikit-learn library. A detailed experiment was performed using 75 % of the dataset for training and the remainder for testing.

Eight ISPD 2016 circuits were selected as experimental subjects, and their specific circuit information is presented in Table 2. The table's columns represent the circuit name, number of timing paths, number of nets, number of cells, timing constraints, number of lookup tables, and number of flip-flops. Strict timing constraints have been set to highlight problems such as timing violations and path delays. By evaluating the timing performance of the circuit under extreme conditions, a better understanding of its limitations allows necessary measures to be taken to improve the design and ensure that the circuit meets the timing requirements in practical applications. These experimental settings ensured an accurate and comprehensive evaluation of the circuit's performance, providing a reliable basis for this study.

4.2. Slack and delay prediction

Tables 3 and 4 present the results of slack and delay prediction using the XGBoost and RF algorithms, along with the improvements achieved by the augmented model. Compared to the basic model, the augmented model shows clear improvements in predicting post-routing path slack and delay.

Specifically, for the augmented model utilizing the XGBoost algorithm, the mean squared error (MSE) for slack prediction decreased by 12.99 %, reaching 0.007590. The R-squared R^2 value increased by 2.19 % to 0.9314, and the correlation coefficient (r) improved by 1.07 % to 0.9425. Furthermore, the augmented model's prediction of delay resulted in a 30.47 % reduction in MSE (to 0.001618), a 0.44 % increase in R^2 (to 0.9857), and a 0.22 % increase in r (to 0.9950).

For the augmented model utilizing the RF algorithm, the mean squared error (MSE) for slack prediction decreased by 12.5 %, reaching 0.004515. The R-squared (R^2) value increased by 1.06 % to 0.9314, and the correlation coefficient (r) improved by 0.54 % to 0.9652. Furthermore, the augmented model's prediction of delay resulted in an 11.69 % reduction in MSE (to 0.000302), a 0.03 % increase in R^2 (to 0.9973), and a 0.02 % increase in r (to 0.9987).

These results confirmed the efficiency of complex network features in predicting post-routing slack and delay. In each prediction, R^2 and r reached a high value (greater than 0.8), indicating that the models achieved high correlation and low bias.

Fig. 4 shows the prediction performance comparison of slack and delay using the metric R^2. As can be seen, the predicted performance of the augmented model is much better than that of the basic model, which indicates the advantage of applying complex network features. Furthermore, delay prediction is better for both models than slack prediction, suggesting that the proposed method and features are more effective for delay prediction.

Table 3
Slack prediction results.

Evaluation metrics	Delay			
	XGBoost		RF	
	Basic model	Augmented model (%)	Basic model	Augmented model (%)
MSE	0.008724	0.007590 (12.99)	0.005160	0.004515 (12.50)
R^2	0.8674	0.8864 (2.19)	0.9216	0.9314 (1.06)
r	0.9325	0.9425 (1.07)	0.9600	0.9652 (0.54)

Table 4
Delay prediction results.

Evaluation metrics	Delay			
	XGBoost		RF	
	Basic model	Augmented model (%)	Basic model	Augmented model (%)
MSE	0.001618	0.001125 (30.47)	0.000342	0.000302 (11.69)
R^2	0.9857	0.9900 (0.44)	0.9970	0.9973 (0.03)
r	0.9928	0.9950 (0.22)	0.9985	0.9987 (0.02)

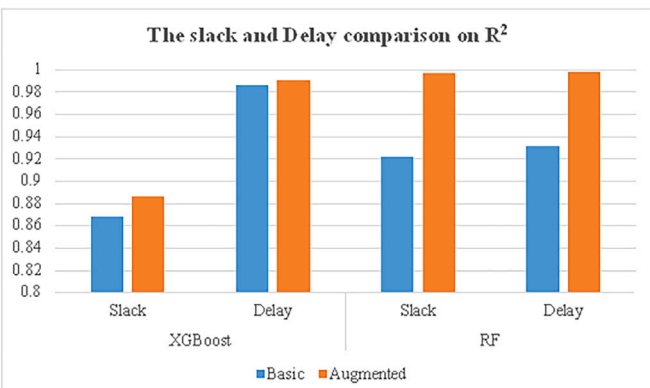

Fig. 4. The slack and delay comparison on R^2.

4.3. TNS and WNS prediction

Worst negative slack (WNS) and total negative slack (TNS) are two critical metrics used in circuit timing analysis to evaluate the quality of timing constraints. WNS represents the least amount of negative slack among all timing paths. When slack is less than zero, it indicates a timing violation. Therefore, WNS represents the most severe instance of timing violations in the entire circuit. TNS, on the other hand, is the sum of the negative slack values across all timing paths, providing an assessment of the overall quality of the timing constraints.

To facilitate future comparisons with similar work [5], we adopt a learning strategy where the sets of circuits are divided into training and testing sets. In particular, circuits D6, D7, and D8 are reserved in the testing set and are not involved in the model training process.

As shown in Table 5, the augmented model with added complex network features demonstrates predictions closer to the actual post-layout values for a broader range of circuits. Here, "deviation" refers to the average deviation between model predictions and actual data for each circuit. Compared to the basic model, the augmented model utilizing the RF algorithm reduces the deviation in predicting WNS by 6.53 %, achieving 5.30 %. The deviation in predicting TNS is reduced by 1.59 %, reaching 12.38 %. This validates the effectiveness of complex network features in predicting post-layout TNS and WNS values.

Fig. 5 illustrates a comparison of the predictive performance of WNS and TNS. As can be seen, the predictions for WNS and TNS using the augmented and basic models are almost identical and close to the real data, except for circuit D7. The TNS prediction for circuit D7 using the basic model deteriorates more than the augmented model. Circuit D7 includes fewer paths and more components, possibly leading to relative inaccuracy. Overall, the results demonstrate the efficiency of the proposed method in predicting WNS and TNS, showing a slight advantage of the augmented model.

4.4. Critical path detection

Identifying critical paths is crucial in static timing analysis. The paths with less than zero slack are considered timing violations and classified as critical paths. Figs. 6 and 7 illustrate the ROC curves for critical path detection using XGBoost and RF algorithms. Both algorithms yield augmented models with improved predictive outcomes compared to the basic model. Specifically, the augmented model employing the XGBoost algorithm achieves an AUC value of 0.9728, a 0.85 % improvement over the basic model. The augmented model utilizing the RF algorithm

Table 5
Predicted results for WNS and TNS.

Critical	WNS (ns)			TNS (ns)		
	Real data	Basic model	Augmented model	Real data	Basic model	Augmented model
D1	−3.82	−3.28	−3.39	−152.01	−128.52	−133.94
D2	−7.32	−6.76	−6.69	−614.07	−575.53	−577.67
D3	−10.88	−9.94	−9.70	−780.81	−735.19	−736.66
D4	−5.90	−5.63	−5.62	−313.03	−280.77	−278.45
D5	−15.49	−14.41	−14.44	−1668.21	−1672.30	−1646.22
D6	−3.69	−2.77	−2.96	−695.93	−572.23	−578.74
D7	−15.57	−10.48	−10.49	−3278.76	−678.44	−2721.68
D8	−7.22	−5.13	−5.32	−829.10	−921.99	−671.98
deviation (%)	–	5.67	5.30	–	12.58	12.38

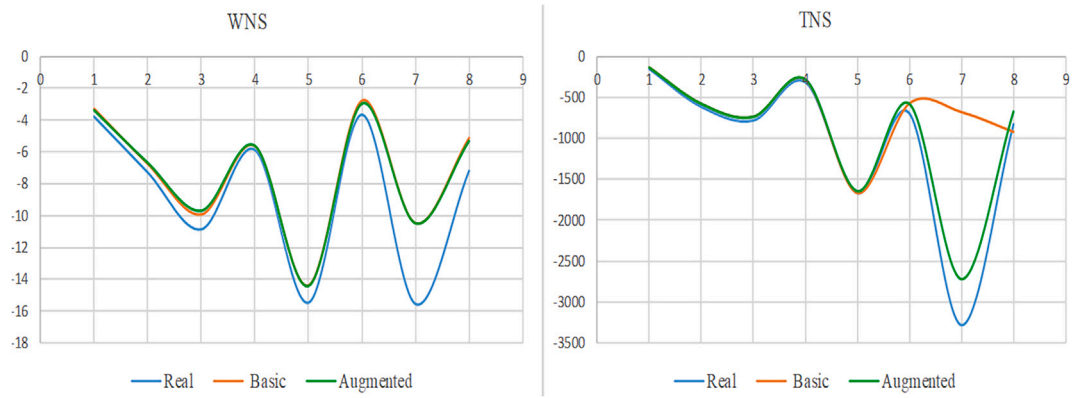

Fig. 5. The WNS and TNS comparison.

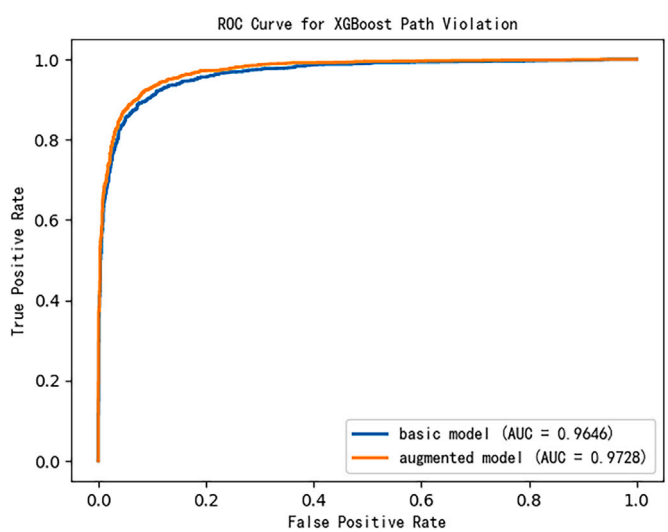

Fig. 6. ROC curve for XGBoost path violation.

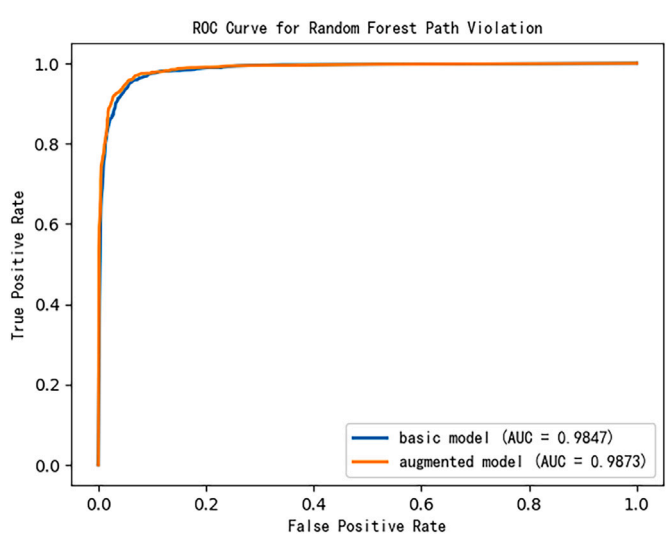

Fig. 7. ROC curve for RF path violation.

achieves an AUC value of 0.9873, a 0.26 % improvement over the basic model. This confirmed the efficacy of complex network features in critical path identification.

Table 6
Comparison with previous work [5].

Assignment	Evaluation metrics	This work	[5]	Improvements
Slack	MSE	0.004463	0.012000	62.81 %
Dely	MSE	0.000302	0.000180	−67.78 %
	r	0.9987	0.9710	2.85 %
TNS	deviation (%)	12.38	187	93.38 %
Violate	AUC	0.9873	0.9700	1.78 %

4.5. Comparison with previous work

As demonstrated in Table 6, a comparison is made between our approach and the method proposed by Barboza et al. [5], which employed the RF algorithm for timing prediction. Compared with their approach, our method exhibits a 62.5 % reduction in MSE for slack prediction. Although our approach's MSE in predicting delay is slightly higher than in their work, the correlation coefficient r demonstrates notably superior performance, with a 2.85 % enhancement.

Furthermore, our method showcases a lighter deviation in predicting TNS, with a 93.38 % improvement and an AUC enhancement of 1.78 % in critical path identification. The experimental results highlight the efficiency of our proposed method across multiple timing tasks, including slack, delay, TNS prediction, and critical path identification. It ensures the effectiveness of the proposed approach.

5. Discussion

To address the uncertainty issue in traditional methods for timing analysis during the placement phase of circuit design, this paper proposes a machine-learning approach that utilizes complex network features to predict post-routing circuit timing. Two models are constructed, including a basic model and an augmented model. Experimental results demonstrate that the augmented model, incorporating complex network features, outperforms the basic model in timing prediction. It highlights the efficiency of complex network features in enhancing the performance of timing prediction models. This approach offers a novel perspective and method for timing optimization in the VLSI design field.

It has been demonstrated that circuit features related to cell and path delay are significant in predicting the timing of analyses in machine learning models. The transition rate determines cell delay by altering transition time, which affects path delay due to uncertainty in arrival time. Therefore, cell delay and transition rate are mutually dependent factors in timing analysis.

The lack of information on cell delay and transition rate may affect the predictive ability of the proposed model. Future work should first investigate the impact on prediction performance of having or not having information on cell delay and transition rate to explore the origin

of the phenomena. Secondly, we should identify alternative circuit features for cell and path delay or develop a simplified calculation method. Combining these with complex network features is expected to enhance the performance of the proposed method.

CRediT authorship contribution statement

Tingyuan Nie: Investigation, Funding acquisition, Conceptualization, Methodology. **Mingzhi Zhao:** Formal analysis, Software, Writing – original draft. **Pengfei Liu:** Software, Validation. **Zhenhao Wang:** Writing – review & editing.

Funding

This work was supported by the National Natural Science Foundation of China (No. 61572269) and the Shandong Province Natural Science Foundation (No. ZR2021MF101).

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

No additional support was received for this work.

Data availability

Data will be made available on request.

References

[1] Q. Yanghua, N. Kapre, H. Ng, et al., Improving classification accuracy of a machine learning approach for FPGA timing closure[C]//2016, in: IEEE 24th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM), IEEE, 2016, pp. 80–83.
[2] N. Kapre, B. Chandrashekaran, H. Ng, et al., Driving timing convergence of FPGA designs through machine learning and cloud computing[C]//2015, in: IEEE 23rd Annual International Symposium on Field-Programmable Custom Computing Machines, IEEE, 2015, pp. 119–126.
[3] J. Bhasker, R. Chadha, Static Timing Analysis for Nanometer Designs: A Practical Approach, Springer Science & Business Media, 2009.
[4] Z. Guo, M. Liu, J. Gu, et al., A timing engine inspired graph neural network model for pre-routing slack prediction, in: Proceedings of the 59th ACM/IEEE Design Automation Conference, 2022, pp. 1207–1212.
[5] E.C. Barboza, N. Shukla, Y. Chen, et al., Machine learning-based pre-routing timing prediction with reduced Pessimism, in: Proceedings of the 56th Annual Design Automation Conference 2019, 2019, pp. 1–6.
[6] H. Chang, S.S. Sapatnekar, Statistical timing analysis considering spatial correlations using a single PERT-Like traversal, in: ICCAD-2003 International Conference on Computer Aided Design, IEEE, 2003, pp. 621–625.
[7] Z. Xie, R. Liang, X. Xu, et al., Preplacement net length and timing estimation by customized graph neural network, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 41 (11) (2022) 4667–4680.
[8] Z. Guo, M. Liu, J. Gu, et al., A timing engine inspired graph neural network model for pre-routing slack prediction, in: Proceedings of the 59th ACM/IEEE Design Automation Conference, 2022, pp. 1207–1212.
[9] H.H. Cheng, I.H.R. Jiang, O. Ou, Fast and accurate wire timing estimation on tree and non-tree net structures, in: 2020 57th ACM/IEEE Design Automation Conference (DAC), IEEE, 2020, pp. 1–6.
[10] L.W. Chen, Y.N. Sui, T.C. Lee, et al., Path-based pre-routing timing prediction for modern very large-scale integration designs, in: 2022 23rd International Symposium on Quality Electronic Design (ISQED), IEEE, 2022, pp. 1–6.
[11] P. Yuanbo, Z. Xuehua, Simulation of hydrological diagnosis model based on small world complex network, Comput. Simul. 39 (4) (2022) 477–481.
[12] C. Suli, D. Dong, Q. Bin, et al., Research on detection of abnormal interaction behavior in directed complex network software, Comput. Simul. 39 (11) (2022) 294–298.
[13] A.A. Rezaei, J. Munoz, M. Jalili, et al., A machine learning-based approach for vital node identification in complex networks, Expert Syst. Appl. 214 (2022) 119086.
[14] R.F. Cancho, C. Janssen, V. Solé R, Topology of technology graphs: small world patterns in electronic circuits, Phys. Rev. E 64 (4) (2001) 046119.
[15] T. Nie, P. Liu, K. Zhao, Z. Wang, Complex network knowledge-based field programmable gate arrays routing congestion prediction, Future Gener. Comput. Syst. 168 (2025) 107776.
[16] T. Nie, Y. Wang, P. Liu, K. Zhao, Z. Wang, Predicting routability of FPGA design by learning complex network images, Expert Syst. Appl. 261 (2025) 125486.
[17] A. Han, Z. Zhao, C. Feng, S. Zhang, Stage-based path delay prediction with customized machine learning Technique, in: Proceedings of the 2021 5th International Conference on Electronic Information Technology and Computer Engineering (EITCE '21), 2021, pp. 926–933.
[18] S. Yang, A. Gayasen, C. Mulpuri, et al., Routability-driven FPGA placement contest, in: Proceedings of the 2016 on International Symposium on Physical Design, 2016, pp. 139–143.
[19] W. Li, S. Dhar, D.Z. Pan, UTPlaceF: a routability-driven FPGA placer with physical and congestion aware packing, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 37 (4) (2017) 869–882.
[20] T. Feist, Vivado design suite, White Pap. 5 (2012) 30.
[21] T. Chen, C. Guestrin, Xgboost: A scalable tree boosting system, in: Proceedings of the 22nd ACM sigkdd international conference on knowledge discovery and data mining 2016, pp. 785–794.
[22] A. Pretorius, S. Bierman, S.J. Steel, A meta-analysis of research in random forests for classification, in: 2016 Pattern Recognition Association of South Africa and Robotics and Mechatronics International Conference (PRASA-RobMech), IEEE, 2016, pp. 1–6.
[23] Z. Xie, R. Liang, X. Xu, J. Hu, C. Chang, J. Pan, Y. Chen, Preplacement net length and timing estimation by customized graph neural network, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 41 (2022) 4667–4680.